Problem-based assignment

# Predicting the battery lifetime of a battery-powered embedded system

This handout defines what must be done and submitted. The firmware template, statechart, hardware documentation (schematic, bill of materials) and detailed tool instructions are provided in the repository: https://github.com/MikroprocesoriaiKTU/battery-lifetime-prediction-lab.

## 1 Objective

Learn to predict and measure the operating time of a battery-powered embedded system: build a timing model of the microcontroller operating modes, estimate the average current with a power simulator, measure the current of the complete system, and justify the differences between prediction and measurement.

## 2 Task and variants

Develop an STM32L433-based embedded system that samples all connected analog sensors and the I2C sensor with sampling period $T_s$, averages the readings over interval $T_v$, and shows the averages in physical units on the OLED display. Determine the operating time until the battery is discharged in two ways: 1) with a power simulator (STM32CubeMX PCC) and 2) by measuring the current consumption (X-NUCLEO-LPM01A). The battery, $T_s$ and $T_v$ are given by the individual variant (Table 1); the sensors are given by the board variant (Table 2).

*Table 1. Assignment variants*

| No. | Battery | Ts, s | Tv, s | No. | Battery | Ts, s | Tv, s | No. | Battery | Ts, s | Tv, s |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **1** | Alkaline AAA | 0.2 | 1 | **21** | Li-MNO2 (CR2477) | 0.2 | 3 | **41** | Alkaline AAA | 0.1 | 2 |
| **2** | Alkaline AA | 0.2 | 2 | **22** | Li-SOCL2 (A3400) | 0.2 | 1 | **42** | Alkaline AA | 0.2 | 3 |
| **3** | Alkaline C | 0.3 | 3 | **23** | Li-SOCL2 (AAA700) | 0.5 | 2 | **43** | Alkaline C | 0.2 | 1 |
| **4** | Li-MNO2 (CR2032) | 0.2 | 1 | **24** | Li-SOCL2 (C9000) | 0.3 | 3 | **44** | Li-MNO2 (CR2032) | 0.2 | 2 |
| **5** | Li-MNO2 (CR2477) | 0.2 | 2 | **25** | Alkaline AAA | 0.1 | 1 | **45** | Li-MNO2 (CR2477) | 0.5 | 3 |
| **6** | Li-SOCL2 (A3400) | 0.5 | 3 | **26** | Alkaline AA | 0.4 | 2 | **46** | Li-SOCL2 (A3400) | 0.2 | 1 |
| **7** | Li-SOCL2 (AAA700) | 0.1 | 1 | **27** | Alkaline C | 0.6 | 3 | **47** | Li-SOCL2 (AAA700) | 0.5 | 2 |
| **8** | Li-SOCL2 (C9000) | 0.1 | 2 | **28** | Li-MNO2 (CR2032) | 0.2 | 1 | **48** | Li-SOCL2 (C9000) | 0.3 | 3 |
| **9** | Alkaline AAA | 0.2 | 3 | **29** | Li-MNO2 (CR2477) | 0.1 | 2 | **49** | Alkaline AAA | 0.1 | 1 |
| **10** | Alkaline AA | 0.1 | 1 | **30** | Li-SOCL2 (A3400) | 0.2 | 3 | **50** | Alkaline AA | 0.5 | 2 |
| **11** | Alkaline C | 0.2 | 2 | **31** | Li-SOCL2 (AAA700) | 0.2 | 1 | **51** | Alkaline C | 0.5 | 3 |
| **12** | Li-MNO2 (CR2032) | 0.2 | 3 | **32** | Li-SOCL2 (C9000) | 0.1 | 2 | **52** | Li-MNO2 (CR2032) | 0.2 | 1 |
| **13** | Li-MNO2 (CR2477) | 0.1 | 1 | **33** | Alkaline AAA | 0.5 | 3 | **53** | Li-MNO2 (CR2477) | 0.2 | 2 |
| **14** | Li-SOCL2 (A3400) | 0.4 | 2 | **34** | Alkaline AA | 0.1 | 1 | **54** | Li-SOCL2 (A3400) | 0.5 | 3 |
| **15** | Li-SOCL2 (AAA700) | 0.5 | 3 | **35** | Alkaline C | 0.5 | 2 | **55** | Li-SOCL2 (AAA700) | 0.25 | 1 |
| **16** | Li-SOCL2 (C9000) | 0.1 | 1 | **36** | Li-MNO2 (CR2032) | 0.5 | 3 | **56** | Li-SOCL2 (C9000) | 0.5 | 2 |
| **17** | Alkaline AAA | 0.5 | 2 | **37** | Li-MNO2 (CR2477) | 0.1 | 1 | **57** | Alkaline AAA | 0.6 | 3 |
| **18** | Alkaline AA | 0.4 | 3 | **38** | Li-SOCL2 (A3400) | 0.2 | 2 | **58** | Alkaline AA | 0.1 | 1 |
| **19** | Alkaline C | 0.1 | 1 | **39** | Li-SOCL2 (AAA700) | 0.2 | 3 | **59** | Alkaline C | 0.2 | 2 |
| **20** | Li-MNO2 (CR2032) | 0.1 | 2 | **40** | Li-SOCL2 (C9000) | 0.2 | 1 | **60** | Li-MNO2 (CR2032) | 0.3 | 3 |

Note: if the assigned battery cannot meet the requirements of the system (e.g., maximum current), select the closest suitable battery and justify the choice.

# 3 Laboratory hardware

The setup is a three-level stack (Fig. 1): the X-NUCLEO-LPM01A power-measurement and supply board (bottom), the NUCLEO-L433RC-P board with the STM32L433RCT6 microcontroller (middle), and the course-specific sensor/display add-on board (top). The add-on carries two analog sensors (NTC thermistors or photoresistors in voltage dividers), one I2C digital sensor and a 0.91" 128×32 OLED display (SSD1306). The schematic is shown in Fig. 2 and the components per board variant in Table 2.

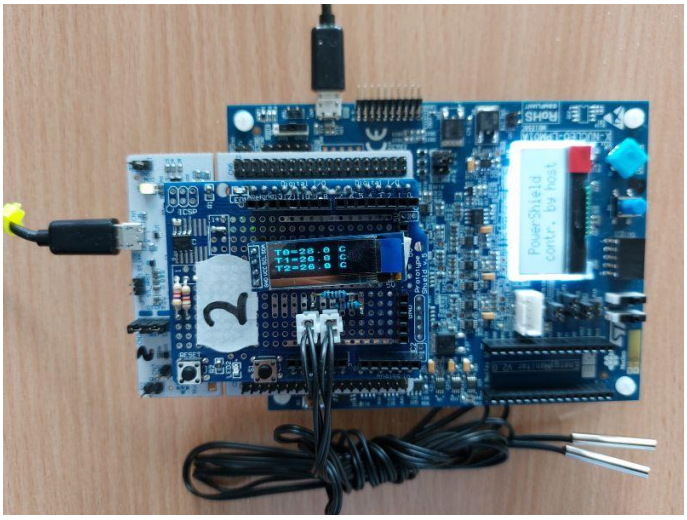


*Fig. 1. Laboratory setup: X-NUCLEO-LPM01A, NUCLEO-L433RC-P and the sensor/display add-on board.*

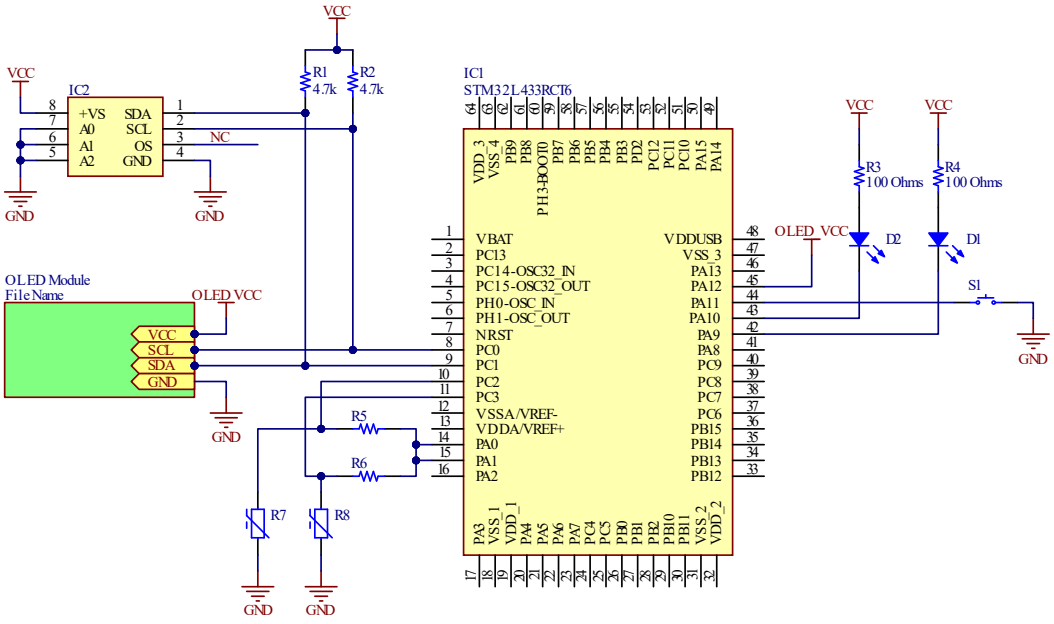



*Fig. 2. Add-on board schematic. PA0 – divider excitation (approx. 3 V) during measurement, PA1 – excitation voltage measurement, PC2 and PC3 – sensor voltages, OLED_VCC – display supply.*

*Table 2. Add-on board components per board variant*

| Board variant | IC1 (digital sensor on I2C3) | R5 and R6 | R7 and R8 (analog sensors) |
|---|---|---|---|
| 1 | MAX7500MSA | 10k | NTC thermistor 10k |
| 2 | MAX7500MSA | 10k | NTC thermistor 10k |
| 3 | MAX7500MSA | 20k | Photoresistor 8-20k (LDR04) |
| 4 | LM75AD | 20k | Photoresistor 8-20k (LDR04) |
| 5 | LM75AD | 20k | Photoresistor 8-20k (LDR04) |
| 6 | LM75AD | 10k | NTC thermistor 10k |
| 7 | BMP180 | 20k | Photoresistor 8-20k (LDR04) |
| 8 | BMP280 | 20k | Photoresistor 8-20k (LDR04) |

Before starting, check the NUCLEO-L433 jumpers: JP4 (nRST) on, all JP5 jumpers off (the board is supplied from X-NUCLEO-LPM01A). The firmware is downloaded from Keil uVision through the Nucleo USB (ST-LINK) port, while the measurement board is controlled through its own USB port (CN5), so nothing has to be reconnected between debugging and measurement.

# 4 Workflow

## 4.1 Firmware template and state machine

The template project (STM32CubeMX configuration, Keil MDK project and itemis CREATE statechart) is provided in the repository folders `firmware/stm32l433/` and `statechart/`. The state machine (Fig. 3) describes one program iteration: the TIM6 interrupt wakes the microcontroller, the I2C sensor is read, the ADC samples the analog channels using DMA, the samples are buffered and, after $T_v$, the averages are displayed. For the rest of the time the microcontroller stays in SLEEP mode.

Verify the statechart with the CREATE simulator, generate the C code and adapt the project to your variant (sensors, $T_s$, $T_v$, display units). Suspend the SysTick timer before entering sleep (`HAL_SuspendTick()`) and resume it in the interrupt handler (`HAL_ResumeTick()`).

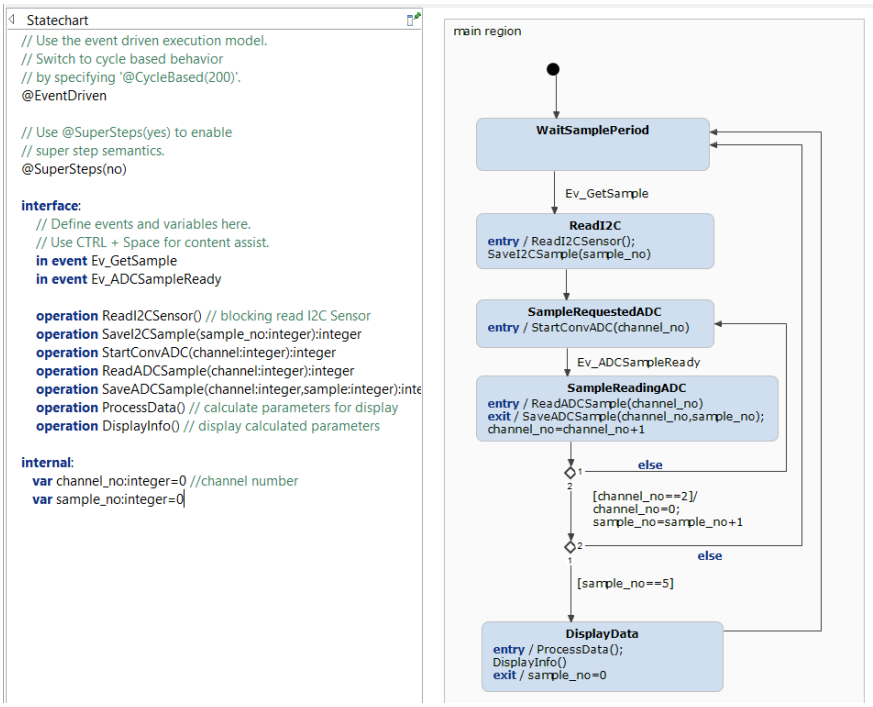



*Fig. 3. Application statechart (itemis CREATE).*

## 4.2 Active and sleep durations

A program iteration is one complete cycle of the state machine. To estimate the average current, the time the microcontroller spends in active (RUN) and sleep (SLEEP) modes during an iteration must be known.

**Problem task No. 1**

Propose a method (other than Keil Event Recorder) to determine the program execution time in the active (RUN) and sleep (SLEEP) modes.

The template uses the Keil MDK Event Recorder (enable it via Manage Run-Time Environment → Compiler → Event Recorder). The `DEBUG_EVENT_…()` macros record timestamps, which are shown in debug mode under View → Analysis Windows → Event Recorder. The active and sleep durations are calculated from the timestamps:

$$T_{act} = (T7 - T2) + (T12 - T8) + (T17 - T13) + (T22 - T18) + (T27 - T23), \quad T_{sleep} = T_{iter} - T_{act}.$$

*Table 3. Example calculation of active and sleep durations from Event Recorder timestamps*

| $T_{iter}$ | 1.0000 | s |
|---|---|---|

| Tstop | time, s | Tstart | time, s | Tstop-Tstart |
|---|---|---|---|---|
| T7 DEBUG_EVENT_GOSLEEP | 0.31732030 | T2 DEBUG_EVENT_TIMER | 0.31637229 | 0.00094801 |
| T12 DEBUG_EVENT_GOSLEEP | 0.51875600 | T8 DEBUG_EVENT_TIMER | 0.51781857 | 0.00093743 |
| T17 DEBUG_EVENT_GOSLEEP | 0.71926101 | T13 DEBUG_EVENT_TIMER | 0.71832359 | 0.00093742 |
| T22 DEBUG_EVENT_GOSLEEP | 0.91976603 | T18 DEBUG_EVENT_TIMER | 0.91882860 | 0.00093743 |
| T27 DEBUG_EVENT_DISPLAY | 1.23116956 | T23 DEBUG_EVENT_TIMER | 1.11933361 | 0.11183595 |

| | | |
|---|---|---|
| $T_{act}$ | 0.11559624 | s |
| $T_{sleep}$ | 0.88440376 | s |

Determine the durations for two compiler optimization settings: -O0 (no optimization) and -Os (balanced).

## 4.3 Average current and battery lifetime model

If the current profile repeats every $T_{iter}$ and, during an iteration, the system spends time $t_m$ in mode m drawing current $I_m$, the average current and the battery lifetime (first-order estimate) are:

$$I_{\text{avg}} = \frac{1}{T_{\text{iter}}} \sum_{m=1}^{M} I_{\text{m}} t_{\text{m}} \quad (1)$$

$$T_{\text{bat}} = \frac{C_{\text{bat}}}{I_{\text{avg}}} \quad (2)$$

where $C_{bat}$ is the usable battery capacity [mAh]. The estimate ignores temperature, self-discharge, battery internal resistance and voltage drop during current pulses – discuss these factors in the report.

## 4.4 Microcontroller current simulation with STM32CubeMX PCC

The STM32CubeMX Power Consumption Calculator (PCC) computes the average microcontroller current and the battery lifetime. Select the assigned battery (Change) and create two power steps (New Step): RUN with duration $T_{act}$ and SLEEP with duration $T_{sleep}$, including the peripherals in use; switch off the Transition checker. PCC models the microcontroller only: estimate the currents of the sensors, display and Nucleo board separately (from datasheets and duty cycles) and add them according to (1).

**Problem task No. 2**

Select the microcontroller clock frequency that minimizes the average current and maximizes the operating time until battery discharge. After changing the frequency, recalculate the TIM6 settings so that $T_s$ and $T_v$ remain unchanged.

## 4.5 Measuring the current of the complete system

The current is measured with X-NUCLEO-LPM01A and STM32CubeMonitor-Power: SELECT BOARD → COMx, TAKE CONTROL; set the supply voltage and sampling rate (e.g., 10 kHz to capture short active-mode peaks) and measure over an integer number of iterations. Show Report displays the average current and

energy. Record every result together with its configuration: variant number, firmware version, clock frequency, optimization, sleep mode, supply voltage and measurement window.

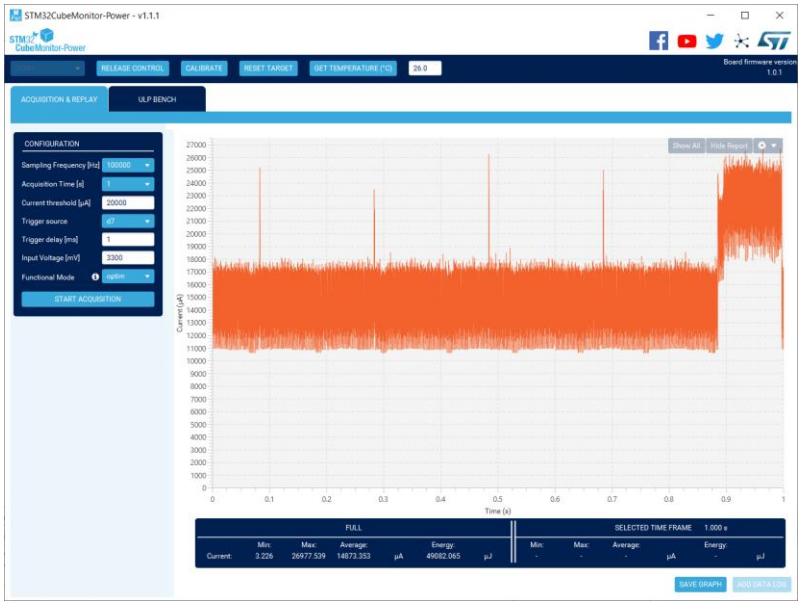



*Fig. 4. Example current profile (80 MHz, SLEEP, average current 14.76 mA): sensor readings and the longer display update are visible within the 1 s iteration.*

## 4.6 Comparison and experiments

Compare the battery lifetimes calculated from the PCC result and from the measured current, and explain the difference component by component (microcontroller, sensors, display, Nucleo board). Report a numerical error only when supply voltage, firmware version and timing window are matched. Then change one parameter at a time: compiler optimization, clock frequency and low-power mode.

**Problem task No. 3**

Implement STOP0 low-power mode instead of SLEEP and estimate how much the battery lifetime increases. Hint: use the RTC clocked from the LSI oscillator to wake up from STOP0.

# 5 Report contents

| Deliverable | Points |
|---|---|
| 1. Project that, according to the individual variant (board sensors, sampling, averaging and display periods), reads the sensors and shows the readings in physical units on the OLED display. | 2 |
| 2. Total active and sleep (SLEEP) time of the microcontroller per iteration, measured with Keil uVision Event Recorder, for two compiler settings: 1) -O0 (no optimization), 2) -Os (balanced). | 2 |
| 3. Average microcontroller current per iteration estimated (simulated) with CubeMX PCC and the corresponding lifetime of the assigned (or selected) battery, for two compiler settings: 1) -O0 (no optimization), 2) -Os (balanced). | 2 |
| 4. Measured average current of the complete system per iteration, obtained with X-NUCLEO-LPM01A. | 1 |

| | |
|---|---|
| 5. Battery lifetime calculated from the measured average current of the setup. | 1 |
| **Problem task No. 1:**<br>Propose a method (other than Keil Event Recorder) to determine the program execution time in the active (RUN) and sleep (SLEEP) modes. | 1 |
| **Problem task No. 2:**<br>Select the microcontroller clock frequency that minimizes the average current and maximizes the operating time until battery discharge. | 1 |
| **Problem task No. 3:**<br>Implement STOP0 low-power mode instead of SLEEP and estimate how much the battery lifetime increases. Hint: use the RTC clocked from the LSI oscillator to wake up from STOP0. | 1 |

# 6 Resources

- Repository (firmware template, statechart, hardware documentation, tool instructions): https://github.com/MikroprocesoriaiKTU/battery-lifetime-prediction-lab
- Keil Event Recorder: https://www.keil.com/pack/doc/compiler/EventRecorder/html/er_use.html
- X-NUCLEO-LPM01A (UM2243): https://www.st.com/en/evaluation-tools/x-nucleo-lpm01a.html
- STM32CubeL4 examples (ADC, PWR): https://github.com/STMicroelectronics/STM32CubeL4/tree/master/Projects/NUCLEO-L433RC-P/Examples
- Battery capacity calculations: https://www.powerstream.com/battery-capacity-calculations.htm
- Capacity of pulsed batteries: https://www.powerstream.com/capacity-of-pulsed-batteries.htm